

Resumen

Corona López José Luis

2024-09-10

Muestreo

Muestreo Simple

```
#install.packages("babynames")
library(babynames)
set.seed(123)
nombres <- sample(babynames$name, 20000, replace = FALSE)
```

Muestreo Sistemático

```
población <- 1500 # Total de individuos
muestra <- 300 # Muestra que queremos
frecuencia <- población/muestra
print(frecuencia)
```

```
## [1] 5
```

```
set.seed(345)
inicio <- sample(1:frecuencia, 1, replace = FALSE)
print(inicio)
```

```
## [1] 5
```

```
secuencia <- seq(inicio, población, frecuencia)
print(secuencia)
```

```
## [1] 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
## [16] 80 85 90 95 100 105 110 115 120 125 130 135 140 145 150
## [31] 155 160 165 170 175 180 185 190 195 200 205 210 215 220 225
## [46] 230 235 240 245 250 255 260 265 270 275 280 285 290 295 300
## [61] 305 310 315 320 325 330 335 340 345 350 355 360 365 370 375
## [76] 380 385 390 395 400 405 410 415 420 425 430 435 440 445 450
## [91] 455 460 465 470 475 480 485 490 495 500 505 510 515 520 525
## [106] 530 535 540 545 550 555 560 565 570 575 580 585 590 595 600
## [121] 605 610 615 620 625 630 635 640 645 650 655 660 665 670 675
```

```
## [136] 680 685 690 695 700 705 710 715 720 725 730 735 740 745 750
## [151] 755 760 765 770 775 780 785 790 795 800 805 810 815 820 825
## [166] 830 835 840 845 850 855 860 865 870 875 880 885 890 895 900
## [181] 905 910 915 920 925 930 935 940 945 950 955 960 965 970 975
## [196] 980 985 990 995 1000 1005 1010 1015 1020 1025 1030 1035 1040 1045 1050
## [211] 1055 1060 1065 1070 1075 1080 1085 1090 1095 1100 1105 1110 1115 1120 1125
## [226] 1130 1135 1140 1145 1150 1155 1160 1165 1170 1175 1180 1185 1190 1195 1200
## [241] 1205 1210 1215 1220 1225 1230 1235 1240 1245 1250 1255 1260 1265 1270 1275
## [256] 1280 1285 1290 1295 1300 1305 1310 1315 1320 1325 1330 1335 1340 1345 1350
## [271] 1355 1360 1365 1370 1375 1380 1385 1390 1395 1400 1405 1410 1415 1420 1425
## [286] 1430 1435 1440 1445 1450 1455 1460 1465 1470 1475 1480 1485 1490 1495 1500
```

```
length(secuencia)
```

```
## [1] 300
```

```
#base_muestra <- base_asegurado[secuencia,] # [Filas que quiero, Columnas que quiero]
```

Muestreo Estratificado

```
set.seed(678)
data <- data.frame(Nombre=sample(babynames$name,1000,replace =FALSE),Region=sample(c("Norte","Centro","Sur"),1000,replace =FALSE))
table(data$Region) #Centro 342, Norte 327, Sur 331
```

```
##
## Centro Norte Sur
## 342 327 331
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
## intersect, setdiff, setequal, union
```

```
data_centro <- data%>% filter(Region== "Centro")
data_norte <- data%>% filter(Region== "Norte")
data_sur <- data%>% filter(Region== "Sur")
posiciones_centro <- sample(1:length(data_centro$Nombre),200, replace=FALSE)
posiciones_norte <- sample(1:length(data_norte$Nombre),200, replace=FALSE)
posiciones_sur <- sample(1:length(data_sur$Nombre),200, replace=FALSE)
#Mostramos la informacion de los sujetos que formaran parte de la muestra
muestra_centro <- data_centro[posiciones_centro,]
```

```

muestra_norte <- data_norte[posiciones_norte,]
muestra_sur <- data_sur[posiciones_sur,]
#Unimos en una sola base de datos
muestra_data <- rbind(muestra_centro, muestra_norte,muestra_sur)

```

Muestreo Estratificado Proporcional (MEP)

```

library(babynames)

set.seed(615)

#Generamos una base de datos

data <- data.frame(Nombre=sample(babynames$name,1000,replace =FALSE),Region=sample(c("Norte","Centro","Sur"),1000,replace=TRUE))

table(data$Region)

##
## Centro  Norte   Sur
##   307    349    344

#Centro 307, Norte 349, Sur 344

C <- 307/1000
N <- 349/1000
S <- 344/1000

Prop_C <- round(C*600,0)
Prop_N <- round(N*600,0)
Prop_S <- round(S*600,0)

library(dplyr)

d_centro <- data%>% filter(Region== "Centro")
d_norte <- data%>% filter(Region== "Norte")
d_sur <- data%>% filter(Region== "Sur")

posiciones_centro <- sample(1:length(d_centro$Nombre),Prop_C, replace=FALSE)
posiciones_norte <- sample(1:length(d_norte$Nombre),Prop_N, replace=FALSE)
posiciones_sur <- sample(1:length(d_sur$Nombre),Prop_S, replace=FALSE)

m_centro <- d_centro[posiciones_centro,]
m_norte <- d_norte[posiciones_norte,]
m_sur <- d_sur[posiciones_sur,]

muestra_data_prop <- rbind(m_centro, m_norte,m_sur)

```

Proporciones

Intervalo de confianza cuando no se tiene a toda la población

```
intervalo_confianza_p <- function(exitos, n, nivel_confianza){  
  p_muestral <- exitos / n  
  q_muestral <- 1 - p_muestral  
  alfa <- 1 - nivel_confianza  
  v_critico <- qnorm(1-alfa/2, lower.tail = TRUE)  
  error <- round(v_critico * sqrt(p_muestral * q_muestral / n), 3)  
  inferior <- p_muestral - error  
  superior <- p_muestral + error  
  cat("Tenemos una confianza del", nivel_confianza, "% de que el intervalo (", inferior, ",", superior, ")  
}
```

Tamaño de muestra cuando no se tiene a toda la población

```
n_proporciones <- function(nivel_confianza, p_muestral, error){  
  alfa <- 1 - nivel_confianza  
  v_critico <- qnorm(1 - alfa/2, lower.tail = TRUE)  
  q_muestral <- 1 - p_muestral  
  n <- ceiling((v_critico^2 * p_muestral*q_muestral)/error^2)  
  print(n)  
}
```

Intervalo de confianza cuando se tiene a toda la población

```
int_conf_prop <- function(exitos, n, nivel_confianza, N){  
  p_muestral <- exitos / n  
  q_muestral <- 1 - p_muestral  
  alfa <- 1 - nivel_confianza  
  v_critico <- qnorm(1 - alfa / 2, lower.tail = TRUE)  
  error <- round(v_critico * sqrt((p_muestral*q_muestral) / n) * sqrt((N-n)/(N-1)), 3)  
  inferior <- p_muestral - error  
  superior <- p_muestral + error  
  cat(inferior, ",", superior)  
}
```

Tamaño de muestra cuando se tiene a toda la población

```
# Función tamaño N  
n_prop <- function(nivel_confianza, p_muestral, error, N){  
  alfa <- 1 - nivel_confianza  
  v_critico <- qnorm(1 - alfa/2, lower.tail = TRUE)  
  q_muestral <- 1 - p_muestral  
  
  n <- N/((error/v_critico)^2*((N-1)/(p_muestral*q_muestral))+1)
```

```
print(n)
}
```

Media

Intervalo de confianza cuando sigma se conoce y no se tiene a toda población

Si no se conoce a sigma, utilizar qt para el v_critico y los grados de libertad serían n-1 y verificar los supuestos

Requisitos: 1. La muestra es aleatoria simple. 2. Cualquiera o ambas de estas condiciones se satisfacen: la población está normalmente distribuida o $n > 30$

```
# Verificar los requisitos:
# 1. La muestra es aleatoria simple
# 2. El valor de la desviación estándar poblacional es conocido
# 3. Cualquiera o ambas de estas condiciones se satisfacen: la población está normalmente distribuida o n > 30

intervalo_media <- function(nivel_confianza, n, media_muestral, sigma){
  alfa <- 1 - nivel_confianza
  v_critico <- qnorm(1 - alfa / 2, lower.tail = TRUE)
  error <- round(v_critico * sigma / sqrt(n), 3)
  inferior <- media_muestral - error
  superior <- media_muestral + error
  cat("El intervalo de confianza para la media poblacional es:", inferior, ", " ,superior,")")
}
```

Tamaño de muestra cuando se conoce sigma y no se tiene a toda la población

Si no se conoce a sigma, utilizar qt para el v_critico y los grados de libertad sería n-1

```
tamaño_muestra <- function(nivel_confianza, sigma, error){
  alfa <- 1 - nivel_confianza
  v_critico <- qnorm(1-alfa / 2, lower.tail = TRUE)
  n <- (v_critico * sigma / error)^2
  cat("El tamaño de muestra requerido es: ",n)
}
```

Intervalo de confianza cuando se tiene a toda población

```
inter_media_N <- function(nivel_confianza, media_muestral, n, N, sigma){
  alfa <- 1 - nivel_confianza
  v_critico <- qnorm(1 - alfa / 2, lower.tail = TRUE)
  error <- round(v_critico * sigma / sqrt(n) * sqrt((N-n)/(N-1)), 3)
  inferior <- media_muestral - error
  superior <- media_muestral + error
  cat("El intervalo de confianza para la media poblacional con tamaño de muestra N es:", inferior, ", " ,superior,")")
}
```

Tamaño de la muestra cuando se tiene a toda la población

```
media_n <- function(nivel_confianza, sigma, error, N){
  alfa <- 1 - nivel_confianza
  v_critico <- qnorm(1 - alfa / 2, lower.tail = TRUE)
  n <- ceiling(((v_critico)^2 * sigma^2 * N)/(error^2 * (N-1) + ((v_critico)^2 * sigma^2)))
  print(n)
}
```

Sigma o Desviación Estandar

Intervalo de confianza

```
# Requisitos:
# La muestra es aleatoria simple
# La población debe tener valores distribuidos normalmente (aún si la muestra es grande)

intervalo_sd <- function(n, nivel_confianza, sd_muestral){
  alfa <- 1 - nivel_confianza
  v_izquierda <- qchisq(alfa / 2, n - 1)
  v_derecha <- qchisq(1 - alfa / 2, n - 1)
  inferior <- sqrt(((n - 1) * sd_muestral^2) / v_derecha)
  superior <- sqrt(((n - 1) * sd_muestral^2) / v_izquierda)
  cat("El intervalo de confianza para estimar a la desviación estándar poblacional es :
      (", inferior,",",superior,")")
}
```

Sigma Cuadrada o Varianza

Intervalo de confianza

```
# Requisitos:
# La muestra es aleatoria simple
# La población debe tener valores distribuidos normalmente (aún si la muestra es grande)

intervalo <- function(n, nivel_confianza, sd_muestral){
  alfa <- 1 - nivel_confianza
  v_izquierda <- qchisq(alfa / 2, n - 1)
  v_derecha <- qchisq(1 - alfa / 2, n - 1)
  inferior <- ((n - 1) * sd_muestral^2) / v_derecha
  superior <- ((n - 1) * sd_muestral^2) / v_izquierda
  cat("El intervalo de confianza para estimar a la varianza poblacional es : (", inferior,",",superior,
  )
}
```